

Reti di Petri (P/T nets)

NECKER

21 Novembre 2025

Indice

1	Introduzione	3
2	Definizione formale (P/T net classica)	3
2.1	Notazione utile	3
3	Dinamica della rete	3
4	Esempio grafico (Produttore-Consumatore-Buffer)	4
5	Relazioni tra transizioni: Sequenza, Conflitto, Concorrenza	4
5.1	1. Sequenza (causalità)	4
5.2	2. Conflitto	4
5.3	3. Concorrenza (parallelismo)	5
5.4	Riassunto delle tre relazioni	5
6	Proprietà principali	5
6.1	Insieme di raggiungibilità e limitatezza	5
6.1.1	Vitalità di una transizione, conservatività e revertibilità	6
7	Estensioni e riduzioni delle reti di Petri classiche	7
8	Analisi delle reti di Petri	8
8.1	Tecniche disponibili	8
8.2	Grafo di raggiungibilità (solo reti limitate)	8
8.3	Grafo di copertura – funziona anche su reti illimitate	9
9	Rappresentazione matriciale	10
9.1	P-invarianti (invarianti di posto)	11
9.2	T-invarianti (invarianti di transizione)	13
9.3	Riepilogo definitivo (la tabella da stampare e appendere al muro)	13
10	Controllo a stati proibiti (controllore a vincoli lineari)	14
10.1	Controllo a stati proibiti (controllore a vincoli lineari)	14
11	Reti temporizzate – Time Basic (TB)	15
11.1	Modelli temporali – panoramica	15
11.2	Evoluzione – le 4 semantiche temporali	17
11.3	Semantica temporale mista (Mixed Time Semantics)	18
12	Le reti Time Basic sono riducibili a reti ad alto livello?	19
12.1	Reti Time Petri ad Alto Livello (HLTPN)	19

13 Analisi di raggiungibilità temporale nelle Time Basic	19
13.1 Stati simbolici $[\mu, C]$ – la soluzione	20
13.2 Formula per generare il nuovo stato simbolico	20
13.3 Dall'albero infinito al grafo finito – le 4 tecniche di contrazione	20
13.4 Riepilogo delle 4 tecniche (tabella da imparare)	23

1 Introduzione

Le **reti di Petri** sono un linguaggio formale per modellare sistemi concorrenti, distribuiti e asincroni. A differenza delle FSM (macchine a stati finiti):

- Lo stato è **distribuito** (marcatore globale = distribuzione di gettoni nei posti)
- Più “stati locali” possono essere attivi contemporaneamente
- Rappresentazione compatta di sistemi con concorrenza e risorse condivise

Utilizzate soprattutto in contesti **critici** (safety-critical) per specifica formale e verifica.

2 Definizione formale (P/T net classica)

Una rete di Petri è una 5-tupla $\mathcal{N} = (P, T, F, W, M_0)$ dove:

Simbolo	Significato
P	Insieme finito di posti (cerchi)
T	Insieme finito di transizioni (rettangoli)
$F \subseteq (P \times T) \cup (T \times P)$	Relazioni di flusso (archi)
$W : F \rightarrow \mathbb{N}^+$	Peso degli archi (1 se non specificato)
$M_0 : P \rightarrow \mathbb{N}$	Marcatore iniziale (numero di gettoni per posto)

Condizioni: $P \cap T = \emptyset$, $P \cup T \neq \emptyset$

2.1 Notazione utile

i preset di una transizione rappresentano i nodi da cui partono le frecce verso la stessa transizione, i postset rappresentano i nodi a cui arrivano le frecce della transizione

- $\bullet x = \text{Pre}(x) = \{y \mid (y, x) \in F\}$ (preset di x)
- $x^\bullet = \text{Post}(x) = \{y \mid (x, y) \in F\}$ (postset di x)
- $M(p)$ = numero di gettoni nel posto p nella marcatura M

3 Dinamica della rete

- Una transizione t è **abilitata** in M (scritta $M[t]$) se

$$\forall p \in \bullet t \quad M(p) \geq W(\langle p, t \rangle)$$

- Lo **scatto** di t produce M' , scritta $M[t]M'$:

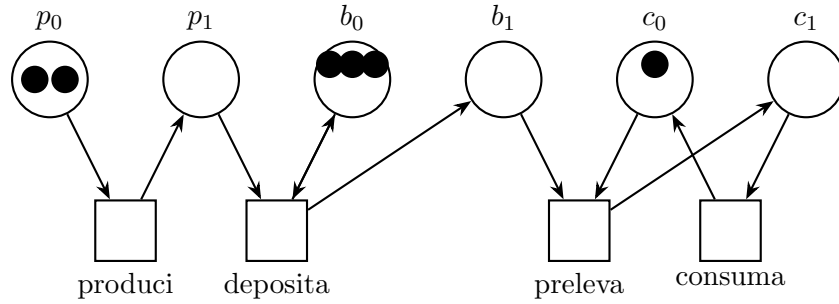
$$M'(p) = \begin{cases} M(p) - W(\langle p, t \rangle) + W(\langle t, p \rangle) & \text{se } p \in \bullet t \cap t^\bullet \\ M(p) - W(\langle p, t \rangle) & \text{se } p \in \bullet t \setminus t^\bullet \\ M(p) + W(\langle t, p \rangle) & \text{se } p \in t^\bullet \setminus \bullet t \\ M(p) & \text{altrimenti} \end{cases}$$

a parole:

1. se il nodo è sia preset che postset
2. se il nodo è solo nei preset
3. se il nodo è solo nei postset
4. se il nodo non è nei postset né nei preset

Proprietà dell'analisi: per sapere se t è abilitata basta guardare solo i posti in $\bullet t$.

4 Esempio grafico (Produttore-Consumatore-Buffer)



5 Relazioni tra transizioni: Sequenza, Conflitto, Concorrenza

Le reti di Petri permettono di classificare il tipo di interazione tra due transizioni t_1 e t_2 . Si distinguono sempre due livelli di analisi:

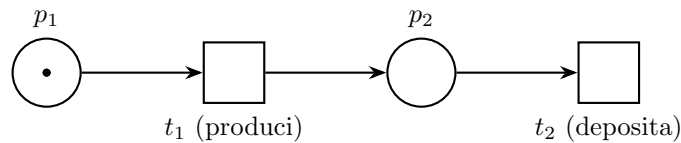
- **Strutturale** dipende solo dalla topologia della rete (archi)
- **Effettiva** dipende dalla marcatura corrente (gettoni presenti)

5.1 1. Sequenza (causalità)

Lo scatto di t_1 abilita t_2 (almeno in certi cammini).

Sequenza effettiva in una marcatura M :

$$M[t_1] \wedge \neg M[t_2] \wedge \exists M' M[t_1]M'[t_2]$$



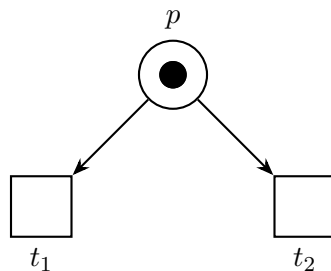
5.2 2. Conflitto

Conflitto strutturale: se due transizioni hanno come preset almeno un nodo in comune

$$\bullet t_1 \cap \bullet t_2 \neq \emptyset$$

Conflitto effettivo in M : se due transizioni in conflitto strutturale sono abilitate ma la somma dei token che richiedono è minore della marcatura attuale in quel nodo

$$M[t_1] \wedge M[t_2] \wedge \exists p \in \bullet t_1 \cap \bullet t_2 : M(p) < W(p, t_1) + W(p, t_2)$$



5.3 3. Concorrenza (parallelismo)

Concorrenza strutturale: contrario di conflitto, se due transizioni non hanno nessun nodo in comune nei loro preset

$$\bullet t_1 \cap \bullet t_2 = \emptyset$$

Concorrenza effettiva in M : se 2 transizioni sono abilitate, se un nodo fa parte del preset di entrambe deve avere abbastanza token per attivarle entrambe.

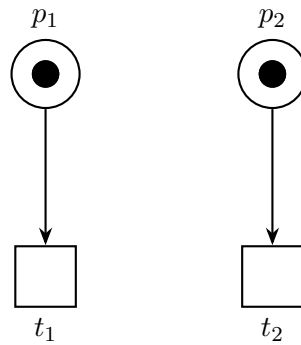
1.

$$M[t_1] \wedge M[t_2]$$

2.

$$\forall p \in \text{Pre}(t_1) \cap \text{Pre}(t_2) \quad M(p) \geq W(\langle p, t_1 \rangle) + W(\langle p, t_2 \rangle)$$

(se sono strutturalmente concorrenti la seconda condizione è automaticamente vera, inoltre possiamo avere varie condizioni es: c. strutturali e effettive, non strutturali ma effettive ecc..)



5.4 Riassunto delle tre relazioni

Relazione	Livello strutturale	Livello effettivo (in marcatura M)
Sequenza	—	$M[t_1] \wedge \neg M[t_2] \wedge \exists M' : M[t_1]M'[t_2]$
Conflitto	$\bullet t_1 \cap \bullet t_2 \neq \emptyset$	$M[t_1] \wedge M[t_2]$ e non abbastanza gettoni per entrambe
Concorrenza	$\bullet t_1 \cap \bullet t_2 = \emptyset$	$M[t_1] \wedge M[t_2]$

Regola pratica:

- **Condividono** almeno un posto in ingresso possibile **conflitto**
- **Non condividono** posti in ingresso possibile **concorrenza**
- Lo scatto di una abilita l'altra **sequenza**

6 Proprietà principali

6.1 Insieme di raggiungibilità e limitatezza

- **Insieme di raggiungibilità** $R(\mathcal{N}, M_0)$ È l'insieme di **tutte** le marcature raggiungibili a partire dalla marcatura iniziale M_0 tramite qualsiasi sequenza (finita) di scatti di transizioni.

Definizione induttiva (la più piccola che soddisfa):

$$M_0 \in R(\mathcal{N}, M_0)$$

$$\text{se } M' \in R(\mathcal{N}, M_0) \text{ e } \exists t \in T \text{ tale che } M'[t]M'' \implies M'' \in R(\mathcal{N}, M_0)$$

In questo caso se esiste una marcatura M' attivata da M_0 ed essa attiva M'' allora M'' fa parte dell'insieme.

In parole semplici:

«parti da M_0 e fai scattare tutte le transizioni che vuoi, nell'ordine che vuoi, quante volte vuoi: tutte le distribuzioni di gettoni che ottieni stanno in $R(\mathcal{N}, M_0)$ ».

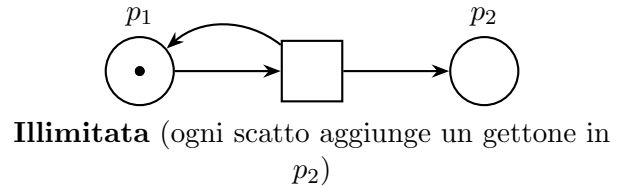
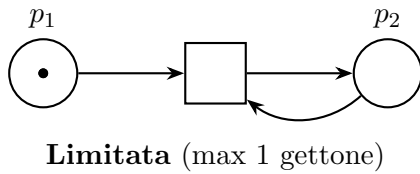
- **Limitatezza** (boundedness) Una rete $(\mathcal{N}, M_0)$ è **limitata** se esiste un numero finito di gettoni che nessun posto potrà mai superare:

$$\exists k \in \mathbb{N} \quad \forall M' \in R(\mathcal{N}, M_0) \quad \forall p \in P \quad M'(p) \leq k$$

Cosa significa praticamente

- Il numero di gettoni in *ogni* posto è sempre limitato superiormente.
- $R(\mathcal{N}, M_0)$ ha cardinalità finita $\iff$ la rete è limitata.
- Se è limitata possiamo costruire il grafo di raggiungibilità (automa equivalente).
- Se *non* è limitata $\Rightarrow$ esiste almeno un posto in cui possiamo pompare infiniti gettoni.

Esempi immediati



- **Conseguenza fondamentale**

rete limitata $\iff |R(\mathcal{N}, M_0)| < \infty \iff$ esiste un automa a stati finiti equivalente alla rete

Per questo la limitatezza è la prima proprietà che si verifica quando si vuole analizzare automaticamente una rete di Petri.

6.1.1 Vitalità di una transizione, conservatività e revertibilità

- **Vitalità di una transizione t :**

- Grado 0 (morta): mai abilitata
- Grado 1: abilitata almeno una volta
- Grado 2: stabilito qualsiasi n positivo, esiste una sequenza di scatti che la fa scattare n volte (posso farla scattare quante volte voglio senza limitazioni, quindi infinite volte ma può essere bloccata da un'altra transizione, quindi può non essere più infinita)
- Grado 3: scatta infinite volte in almeno una esecuzione (la due non garantisce l'infinità sempre)
- Grado 4 (viva in maniera assoluta): se da ogni marcatura raggiungibile esiste una sequenza che la fa scattare almeno una volta

- **Conservatività** (proprietà di conservazione pesata) Sia $H : P \rightarrow \mathbb{N}^+$ una funzione che assegna ad ogni posto un peso positivo (es. $H(p) = 5$ se p rappresenta 5 unità di una risorsa).

La rete $(\mathcal{N}, M_0)$ è **conservativa** rispetto a H se il valore totale pesato dei gettoni è invariante:

$$\forall M' \in R(\mathcal{N}, M_0) \quad \sum_{p \in P} H(p) \cdot M'(p) = \sum_{p \in P} H(p) \cdot M_0(p) := k$$

Significato intuitivo: ogni transizione distrugge e crea gettoni in modo che il “costo totale pesato” del sistema non cambi mai (la somma pesata di ogni marcatura è uguale).

Conseguenza fondamentale:

$$\text{conservativa} \implies \text{limitata}$$

- **Conservatività stretta** Caso particolare in cui tutti i posti hanno peso 1:

$$H(p) = 1 \quad \forall p \in P$$

Allora la condizione diventa:

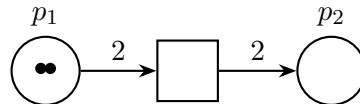
$$\forall M' \in R(\mathcal{N}, M_0) \quad \sum_{p \in P} M'(p) = \sum_{p \in P} M_0(p)$$

cioè il **numero totale di gettoni nella rete è costante** per sempre.

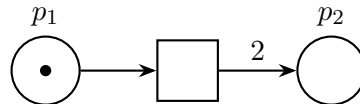
Equivale a dire che **ogni transizione consuma esattamente quanti gettoni produce**:

$$\forall t \in T \quad \sum_{p \in \bullet t} W(\langle p, t \rangle) = \sum_{p \in t \bullet} W(\langle t, p \rangle)$$

- **Esempi grafici**



Conservativa stretta (sempre esattamente 2 gettoni)



Conservativa (con $H(p_1) = 2$ e $H(p_2) = 1$) ma **non** stretta stretta

- **Relazioni di implicazione**

$$\text{conservativa stretta} \implies \text{conservativa} \implies \text{limitata}$$

Le implicazioni inverse **non** valgono.

- **Revertibilità:**

Una rete è revertibile se da qualsiasi marcatura si può tornare alla marcatura di base

7 Estensioni e riduzioni delle reti di Petri classiche

- **Capacità finita dei posti** Ogni posto p ha una capacità massima $C(p)$. La transizione t può scattare solo se dopo lo scatto nessun posto in uscita supera la propria capacità. **Equivalenza:** sì, si traduce in P/T classica aggiungendo un posto complementare con $C(p)$ gettoni iniziali (funziona solo per reti pure, cioè senza archi bidirezionali tra lo stesso posto e la stessa transizione).

- **Archii inibitori** (cerchietto sull'arco) La transizione è abilitata solo se il posto di partenza ha meno di n gettoni (di solito $n=0$). **Equivalenza con P/T classica:**
 - se il posto è limitato $\rightarrow$ sì (posto complementare)
 - se il posto è illimitato $\rightarrow$ no, la rete diventa più potente (Turing-complete)
- **Pesi sugli archi** ($W > 1$) ogni rete con transizioni con pesi superiori a 1 può essere riscritta molto più verbosamente come una rete equivalente con pesi = 1
- **Reti Condizione/Evento (C/E)** Restrizione molto forte:
 - tutti gli archi hanno peso 1
 - ogni posto ha capacità massima 1 (condizione booleana)

Equivalenza: ogni rete P/T **limitata** si può tradurre in una rete C/E equivalente. Le reti C/E non possono rappresentare stati infiniti.
- **Altre estensioni molto usate** (solo nomi)
 - Reti colorate (Colored Petri Nets)
 - Archi di reset
 - Transizioni con priorità
 - Reti temporizzate / stocastiche

8 Analisi delle reti di Petri

Obiettivo: rispondere **prima di scrivere codice** a domande tipiche su sistemi concorrenti/distribuiti:

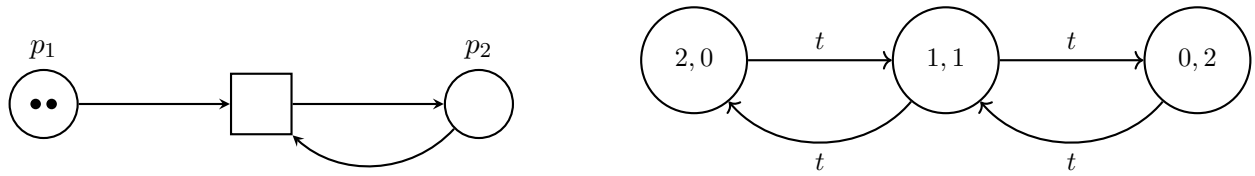
- Esiste una certa marcatura raggiungibile?
- È possibile una certa sequenza di scatti?
- Esiste deadlock?
- La rete (o una transizione) è viva? Di che livello?
- La rete è limitata? Reversibile? Coperta da invarianti?

8.1 Tecniche disponibili

Tecniche dinamiche (esplorano stati)	Tecniche statiche (solo topologia)
Grafo/Albero di raggiungibilità	P-invarianti (invarianti di posto)
Grafo/Albero di copertura (coverability)	T-invarianti (invarianti di transizione)
	Rappresentazione matriciale

8.2 Grafo di raggiungibilità (solo reti limitate)

- **Cosa rappresenta:** ogni possibile configurazione di gettoni raggiungibile partendo da M_0 .
- **Nodi** $\rightarrow$ marcature raggiungibili $M \in R(M_0)$
- **Archii** $\rightarrow M \xrightarrow{t} M'$ se e solo se $M[t]M'$
- È la macchina a stati finiti (FSM) equivalente alla rete.



Proprietà verificabili immediatamente

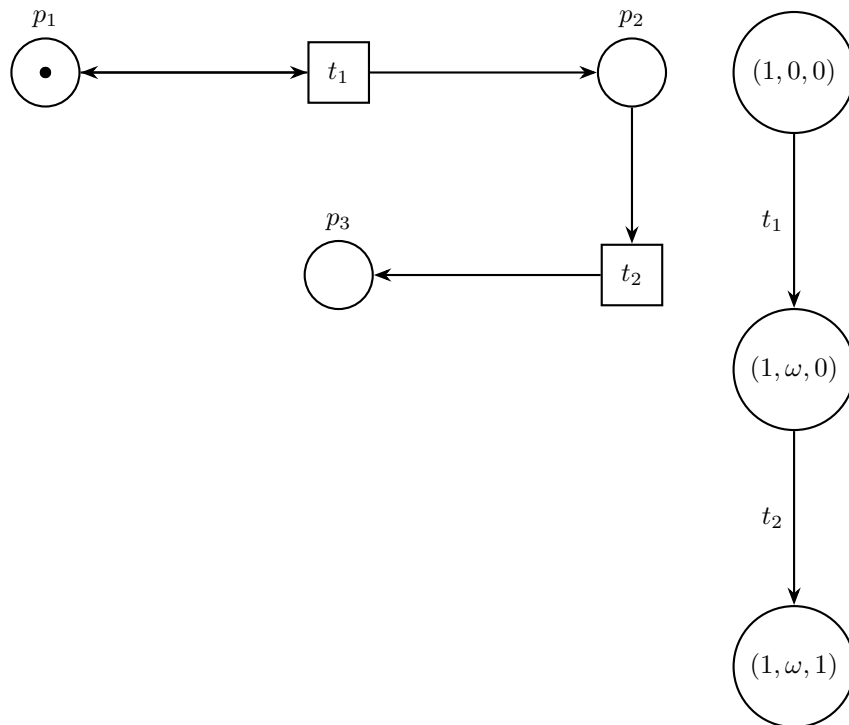
- **Deadlock** → nodo senza archi uscenti
- **Liveness** → ogni transizione appare come etichetta
- **Reversibilità** → grafo fortemente connesso
- **Mutua esclusione** → nessun nodo con $M(p_i) + M(p_j) > 1$

8.3 Grafo di copertura – funziona anche su reti illimitate

Quando la rete è illimitata il grafo di raggiungibilità ha infiniti nodi. Il grafo di copertura comprime infinite marcature simili usando ω («infinito controllato»).

M copre M' $\iff \forall p M(p) \geq M'(p)$

Differenza chiave nell'algoritmo Quando si genera una nuova marcatura M' , se sul cammino dalla radice esiste M'' con $M' \succeq M'' \rightarrow$ mettiamo ω dove $M'(p) > M''(p)$.



Perché questo esempio è corretto (passo passo)

Marcatura iniziale: $M_0 = (1, 0, 0)$

1. Da $(1, 0, 0)$ è abilitata solo $t_1 \rightarrow$ scatta t_1 :

- consuma 1 da p_1
- produce 1 in p_2
- produce 1 in p_1

→ nuova marcatura $(1, \omega, 0)$ questo perché:

- $M_1|1 = 1 \geq 1$ (M_0) (marcatore 1 posto 1 = 1)
- $M_1|2 = 1 > 0$ (M_0) quindi dato che esiste $1 \geq 1$ (nel posto 1), si mette ω
- $M_1|3 = 0 \geq 0$ (M_0)

a parole:

- Guarda il Posto 2: "Ehi, qui sono passato da 0 a 1. È un aumento! Potrebbe essere un infinito?"
- controlla gli altri posti (es. il Posto 1): "Per essere un vero infinito, non devo aver 'pagato' questo aumento svuotando altri posti. Vediamo il Posto 1... ah, c'era 1 e c'è ancora 1 ($1_i=1$). Ottimo, il Posto 1 non è peggiorato."

rissumendo bisogna seguire due regole:

1. verifico la marcatura attuale confrontandola con TUTTE le marcature verso la radice risalendo.
2. per ogni posto se non è peggiorato ma è rimasto almeno uguale e almeno 1 è migliorato, allora in quelli migliorati si mette W.

Cosa possiamo concludere con certezza da questa analisi

- Non appare nessun $\omega \implies$ la rete è limitata (il grafo di copertura coincide con il grafo di raggiungibilità)
- Appare $\omega \implies$ esiste almeno un posto illimitato (qui p_1 è illimitato)
- **Non possiamo dimostrare la liveness** (la presenza di ω nasconde informazioni: una transizione può sembrare abilitata all'infinito senza esserlo davvero, infatti se un nodo con una transizione che genera infiniti token ha anche una transizione che glie li leva e basta, il postset della prima transazione non sarà più ω ma il grafo non è in grado di tracciarlo)
- Ogni marcatura senza ω nel grafo di copertura è sicuramente raggiungibile dalla marcatura iniziale

Regola d'oro

Grafo di raggiungibilità $\rightarrow$ analisi completa (solo se limitata)

Grafo di copertura $\rightarrow$ analisi parziale ma sempre terminante

9 Rappresentazione matriciale

Invece di disegnare la rete, la descriviamo con numeri: è il modo più potente per fare analisi automatica.

- $\mathbf{I} \in \mathbb{N}^{|P| \times |T|}$ **matrice di input (pre-condizioni), P : Posti T : Transizioni**

$$I[i, j] = \text{peso dell'arco } p_i \rightarrow t_j \quad (0 \text{ se l'arco non esiste})$$

- $\mathbf{O} \in \mathbb{N}^{|P| \times |T|}$ **matrice di output (post-condizioni)**

$$O[i, j] = \text{peso dell'arco } t_j \rightarrow p_i \quad (0 \text{ se l'arco non esiste})$$

- $\mathbf{C} = \mathbf{O} - \mathbf{I}$ **matrice di incidenza** (la più importante!)

$$C[i, j] = \begin{cases} +w & \text{se lo scatto di } t_j \text{ aggiunge } w \text{ gettoni in } p_i \\ -w & \text{se lo scatto di } t_j \text{ consuma } w \text{ gettoni da } p_i \\ 0 & \text{altrimenti (nessun arco o arco bidirezionale con stesso peso)} \end{cases}$$

- $\mathbf{m} \in \mathbb{N}^{|P|}$ **vettore colonna della marcatura corrente**

$m[i]$ = numero di gettoni attualmente nel posto p_i

- $\mathbf{s} \in \mathbb{N}^{|T|}$ **vettore di conteggio scatti** (parascatti)

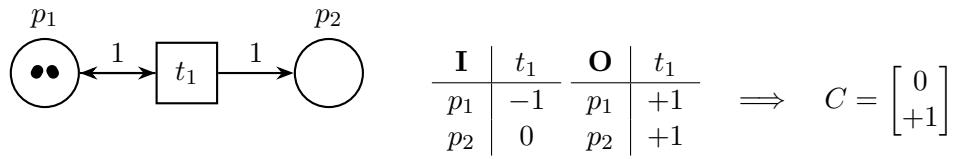
$s[j]$ = quante volte la transizione t_j ha scattato nella sequenza

Equazione di stato (equazione di evoluzione):

$$\boxed{m' = m + Cs}$$

Significato: la nuova marcatura è quella vecchia più l'effetto netto della sequenza di scatti s .

Esempio concreto:



9.1 P-invarianti (invarianti di posto)

Un **P-invariante** è un vettore riga $h \in \mathbb{Z}^{|P|}$, non nullo, che soddisfa

$$\boxed{h^T C = 0}$$

Questa è una condizione **sufficiente** e **necessaria** per essere h un **P-invariante**.

Significato fisico

Il numero $h^T m$ rappresenta una **quantità pesata di gettoni** ed è **invariante** durante tutta l'esecuzione della rete:

$$\boxed{\forall m' \text{ raggiungibile da } m_0 \implies h^T m' = h^T m_0 = \text{costante}}$$

Esempi reali

P-invariante	Rete	Cosa significa fisicamente
$h = [1 \ 1 \ 0 \ 0 \ 0]$	Lettori/Scrittori	LettoriPronti + LettoriAttivi = costante
$h = [0 \ 0 \ 1 \ 1]$	Scrittori	ScrittoriPronti + ScrittoriAttivi = costante
$h = [1 \ 0 \ 1]$	Semaforo	Posti "libero" + "occupato" = 1 sempre
$h = [1 \ 1 \ 1]$	Sistema conservativo	Numero totale di gettoni costante

Teoremi

Condizione sui P-invarianti	Conseguenza (dimostrata)
Esiste $h \geq 0$ (semipositivo)	Tutti i posti con peso $h(i) > 0$ sono limitati
Esiste $h > 0$ (tutti pesi strettamente positivi)	La rete è conservativa $\Rightarrow$ limitata
Esiste un insieme di P-invarianti semi-positivi tali che ogni posto abbia peso > 0 in almeno uno di essi	La rete è limitata (teorema di copertura dei P-invarianti)

Come si trovano i P-invarianti in pratica?

Algoritmo di Farkas – passo passo

Sia C la matrice di incidenza ($|P| \times |T|$).

1. Costruisci la matrice iniziale (concatenando la matrice identità a destra di C)

$$D_0 = [C \quad I_{|P|}]$$

2. Per ogni colonna $j = 1, \dots, |T|$ della parte C :

- Prendi tutte le coppie di righe r_i, r_k di D_{j-1} tali che $D_{j-1}[i, j] \cdot D_{j-1}[k, j] < 0$ (righe che possono essere combinate linearmente)
- Per ogni coppia:

aggiungi in fondo alla tabella una nuova riga $= |D_{j-1}[i, j]| \cdot r_k + |D_{j-1}[k, j]| \cdot r_i$

(risultato della combinazione lineare) Divisa per l'MCD dei suoi elementi (per tenere i numeri piccoli)

3. Elimina dalla matrice tutte le righe che **ancora** hanno un valore non nullo nella colonna j
4. Procedi iterativamente fino ad esaurire la matrice C
5. Alla fine ottieni $D_{|T|}$. **Le ultime $|P|$ colonne di $D_{|T|}$ contengono una base di P-invarianti semi-positivi**

Esempio classico

Rete con matrice di incidenza:

$$C = \begin{bmatrix} -1 & 1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix} \quad (3 \text{ posti, } 2 \text{ transizioni})$$

Applichiamo Farkas:

$$D_0 = \begin{bmatrix} -1 & 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 1 & 0 \\ -1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Colonna 1: coppie (1,2) e (2,3) $\rightarrow$ nuove righe: (0,0,1,1,0) e (0,0,0,1,1) Dopo eliminazione e normalizzazione $\rightarrow$ passo successivo...

Alla fine (quando C è esaurita) si ottiene come soluzioni:

$$h = [1 \ 1 \ 0]$$

$$h = [1 \ 0 \ 1]$$

Riassunto lampo dell'algoritmo di Farkas

Obiettivo	Trovare P-invarianti semi-positivi
Input	Matrice di incidenza C
Passo chiave	Combinazioni lineari di righe con segni opposti
Risultato finale	Base di P-invarianti ≥ 0 nelle ultime colonne
Conclusione	Se ogni posto ha peso > 0 in almeno un invariante $\rightarrow$ rete limitata

l'algoritmo di Farkas è utile perché cerca direttamente gli invarianti semi-positivi e mi permette di concludere subito la limitatezza della rete senza costruire il grafo di copertura

9.2 T-invarianti (invarianti di transizione)

Un **T-invariante** è un vettore colonna $x \in \mathbb{N}^{|T|}$, non nullo, che soddisfa

$$Cx = 0$$

Significato fisico

Esiste una sequenza di scatti il cui **effetto netto è zero**:

$$m + Cx = m \quad \Rightarrow \quad \text{se la sequenza è eseguibile, torni esattamente alla marcatura iniziale}$$

(indica la quantità delle volte che ogni singola transizione deve scattare ma non l'ordine)

Teorema fondamentale

Teorema di vitalità (o di copertura dei T-invarianti)

Se una rete è:

1. **limitata**
2. **coperta da T-invarianti** (cioè esiste una combinazione lineare a coefficienti positivi di T-invarianti che dà un vettore strettamente positivo)

$\Rightarrow$ la rete è **viva** (ogni transizione può scattare infinitamente spesso da ogni stato raggiungibile).

Come si calcolano

Risolviamo $Cx = 0 \rightarrow$ cerchiamo i possibili valori che annullano l'effetto delle transizioni di C .

9.3 Riepilogo definitivo (la tabella da stampare e appendere al muro)

Caratteristica	P-invariante h	T-invariante x
Equazione	$h^T C = 0$	$Cx = 0$
Tipo di vettore	riga	colonna
Cosa rimane costante	quantità pesata di gettoni	sequenze cicliche
Se esiste uno strettamente positivo	rete conservativa $\rightarrow$ limitata	conservativa solo se anche limitata
Se coprono tutto lo spazio	rete limitata	rete viva (se anche limitata)
Utile per dimostrare	limitatezza, mutua esclusione	liveness, correttezza protocolli

ATTENZIONE:

- l'esistenza di un **P-invariante strettamente positivo** è condizione sufficiente e necessaria per far sì che la rete sia conservativa.
- L'esistenza di un **T-invariante** è condizione necessaria ma **NON** sufficiente per dimostrare la **validità** dato che per farlo dobbiamo essere sicuri che la sequenza di scatti sia ammissibile. (il t-invariante ci dice che se scattano tutte allora si ritorna allo stato di partenza, ma non ci dice se quelle transizioni possono scattare perché i gettoni ce lo permettono)

10 Controllo a stati proibiti (controllore a vincoli lineari)

Obiettivo: ****impedire alla rete di raggiungere marcature indesiderate**** (stati proibiti) aggiungendo solo posti e archi (senza toccare le transizioni originali).

Idea di base

Vogliamo imporre che certe combinazioni lineari di gettoni **non superino mai una soglia**:

$$Lm \leq b$$

dove

- L = P-invariante detto vincolo
- m = marcatura corrente
- b = numero di soglia massima (somma pesata totale dei gettoni della rete)

10.1 Controllo a stati proibiti (controllore a vincoli lineari)

Obiettivo: impedire alla rete di entrare in marcature vietate del tipo

$$Lm \leq b$$

Come abbiamo visto in Ricerca Operativa, ogni sistema di disequazioni lineari può essere trasformato in un sistema di equazioni introducendo **variabili di slack** non negative:

$$Lm + x = b \quad x \geq 0$$

Ogni componente di x diventa un **posto di controllo** (posto monitor, posto slack).

Idea

Il posto di controllo p_c contiene inizialmente $b - Lm_0$ gettoni e, ogni volta che il sistema si avvicina al confine del vincolo, il posto di controllo perde gettoni. Quando arriva a 0, **blocca** le transizioni che violerebbero il vincolo.

Le due formule da sapere

Sia C_s la matrice di incidenza della rete originale.

$$C_c = -LC_s \quad (C_c \text{ è la matrice } C \text{ con la riga del nuovo posto } p_c) \quad (1)$$

$$m_{0c} = b - Lm_{0s} \quad (\text{marcatura iniziale dei nuovi posti di controllo}) \quad (2)$$

m_{0s} indica la marcatura iniziale senza il posto di controllo

Queste due formule funzionano sempre, per qualsiasi vincolo lineare $Lm \leq b$.

Esempio 1 – Mutua esclusione semplice

Vogliamo che due processi non siano mai contemporaneamente in zona critica:

$$m(p_1) + m(p_3) \leq 1$$

Vincolo: $L = \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix}$, $b = 1$

Supponiamo

$$C_s = \begin{bmatrix} 0 & -1 & 1 & 0 \\ 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & -1 \\ -1 & 0 & -1 & 1 \end{bmatrix}$$

Allora:

$$C_c = -LC_s = \begin{bmatrix} 0 & 0 & -1 & 1 \end{bmatrix} \quad m_{0c} = 1 - Lm_{0s} = 1$$

cioè:

- C_c indica come si deve comportare il complementare in modo da rispettare $M(p_s) + M(p_c) = b$ (attenzione, è $L1 \cdot C11 + L2 \cdot C21 + L3 \cdot C31 + L4 \cdot C41$, $L1 \cdot C21 + L2 \cdot C22 \dots$)
- m_{0c} indica la marcatura iniziale del posto p_c (e si colloca vincolo $b - L \cdot$ marcatura iniziale)

Tabella riassuntiva

Compito	Cosa fare
Imporre $Lm \leq b$	Aggiungere posti di controllo
Calcolare gli archi	$C_c = -LC_s$
Calcolare gettoni iniziali	$m_{0c} = b - Lm_{0s}$
Risultato	Il vincolo è sempre rispettato e il comportamento è massimamente permissivo

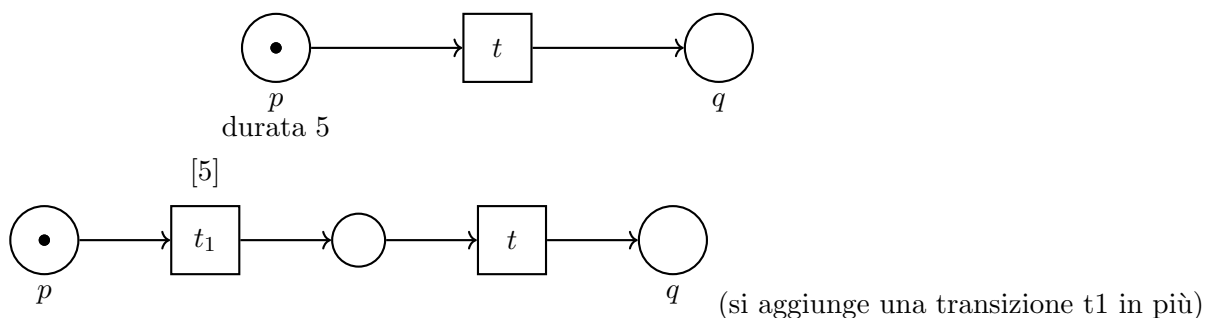
11 Reti temporizzate – Time Basic (TB)

Motivazione: sistemi **Hard Real-Time** in alcuni casi se un vincolo temporale non è rispettato possono succedere catastrofi quindi serve modello deterministico con un tempo esplicito → **Time Basic Petri Nets**

11.1 Modelli temporali – panoramica

Tempo su	Posti	Transizioni
Significato	durata minima di permanenza	durata o istante di scatto
Equivalenza	Sì – trasformabili	

Esempio di equivalenza (tempo su posto → tempo su transizione)



Concetti fondamentali – spiegazione dettagliata

1. **I gettoni sono distinguibili e portano un timestamp** Ogni gettone creato ha un valore $\tau_i \in \Theta$ (solitamente $\mathbb{R}_{\geq 0}$ o $\mathbb{N}$) che rappresenta l'istante esatto di creazione. Due gettoni con timestamp diverso sono **distinti**, anche se stanno nello stesso posto.

2. **Tempo di abilitazione (di una transazione)**

$$\boxed{en(t) = \max\{\tau_1, \tau_2, \dots, \tau_k\}}$$

dove $\tau_1, \dots, \tau_k$ sono i timestamp dei gettoni scelti per formare la tupla abilitante di t . *Intuizione:* la transizione deve aspettare che arrivino **tutti** i gettoni necessari.

3. **Funzione temporale tf_t** Ogni transizione t ha una funzione

$$tf_t : \Theta^k \longrightarrow 2^\Theta$$

che restituisce l'insieme dei tempi di scatto possibili. Nella pratica si usa quasi sempre un intervallo chiuso:

$$tf_t(en) = [eft_t(en), lft_t(en)] \quad (lft_t \text{ può essere } \infty)$$

con la condizione obbligatoria

$$\boxed{eft_t(en) \geq en(t)}$$

4. **Regola di scatto** Una transizione abilitata t può scattare in un qualsiasi istante

$$\theta \in [eft_t(en), lft_t(en)]$$

Tutti i gettoni prodotti da quello scatto avranno **esattamente** timestamp θ .

5. **Marcatura iniziale m_0** Non è più un vettore di interi, ma un multiset di coppie (τ, n) per ogni posto:

$$m_0(p_1) = \{(0, 3), (5, 1)\} \quad (3 \text{ gettoni creati al tempo } 0, 1 \text{ gettone al tempo } 5)$$

Definizione formale

$$\text{Rete Time Basic} = \langle P, T, \Theta, F, tf, m_0 \rangle$$

- P, T, F identici alle reti di petri normali
- Θ è il dominio temporale, ovvero l'insieme numerico che contiene le rappresentazioni degli istanti di tempo
- tf è una funzione che associa ad ogni transizione t appartenente a T una funzione temporale tft che data in input la tupla abilitante en , ovvero l'insieme dei timestamp dei gettoni scelti per l'abilitazione nel preset, restituisce un insieme di tempi di scatto possibili: dove $tf : T \rightarrow (\Theta^k \rightarrow 2^\Theta)$.
- m_0 marcatura iniziale ma con gettoni con info del timestamp

Esempi di funzioni temporali più comuni

Tipo	Notazione	Significato fisico
Fissa	$[3, 3]$	scatta esattamente dopo 3 unità
Ritardo minimo	$[2, \infty)$	può scattare dopo almeno 2 unità (debole)
Finestra	$[4, 8]$	deve scattare tra 4 e 8 unità (es: attiva a 10 scatta tra 14 e 18)
Relativa	$[en + 1, en + 5]$	finestra che si sposta con l'abilitazione
Assoluta	$[10, 15]$	scatta solo tra tempo assoluto 10 e 15

Esempio pratico

Rete: $p_1 \xrightarrow{t [2,5]} p_2$ con due gettoni in p_1 creati al tempo 0.

- $en(t) = \max(0, 0) = 0$
- t può scattare in qualsiasi $\theta \in [2, 5]$
- Supponiamo $\theta = 4 \rightarrow$ il gettone creato in p_2 avrà timestamp 4
- Se esiste t_2 che consuma p_2 con $tf_{t_2} = [\theta + 1, \theta + 3]$, allora t_2 potrà scattare in $[5, 7]$

11.2 Evoluzione – le 4 semantiche temporali

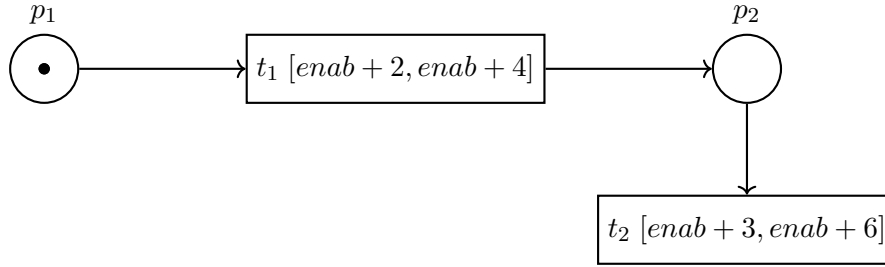
Semantica	A1	A2	A3	A4	A5	Comportamento delle transizioni
WTS (debole)	✓		✓			possono non scattare mai
MWTS (monotonica debole)	✓	✓	✓			possono non scattare
STS (forte)	✓	✓	✓	✓	✓	devono scattare entro lft
Mista	transizioni marcate “W” = deboli					transizioni senza “W” = forti

- **A1 – Monotonicità rispetto alla marcatura iniziale** Tutti i tempi di scatto futuri devono essere $\geq$ del massimo timestamp presente nella marcatura iniziale (dei gettoni che la transazione consuma). *Non si possono creare gettoni nel passato.* **Esempio:** se all’avvio c’è un gettone con $\tau = 10$, nessuna transizione che necessita quel gettone, potrà mai scattare prima del tempo 10.
- **A2 – Monotonicità della sequenza di scatto** I tempi di scatto nella sequenza devono essere non decrescenti: $\theta_1 \leq \theta_2 \leq \theta_3 \leq \dots$. *Il tempo globale della rete non può mai tornare indietro.* **Esempio:** non è ammessa la sequenza t_1 a 12 $\rightarrow t_2$ a 8 $\rightarrow t_3$ a 15.
- **A3 – Non-Zenone (divergenza del tempo)** Non è possibile avere infiniti scatti in un intervallo di tempo finito. *Niente cicli istantanei infiniti.* **Esempio:** una transizione con $[0, 0]$ (tempo min, tempo max di scatto) in un ciclo non è ammessa.
- **A4 – Marcatura iniziale forte** Nella marcatura iniziale, per ogni transizione già abilitata, il suo lft (Latest Firing Time) deve essere $\geq$ del massimo timestamp iniziale. *Non può esistere una transizione che ha un lft minore del timestamp di qualsiasi gettone che l’abilita (a diff di A1, questa garantisce la validità della condizione di partenza, a1 invece garantisce che lo scatto avvenga dopo).* **Esempio:** gettone con $\tau = 15$ e transizione abilitata con $lft = 12 \rightarrow$ marcatura iniziale non valida in STS.
- **A5 – Sequenza di scatto forte** Se una transizione scatta al tempo θ , non deve esistere nessuna altra transizione abilitata con $lft < \theta$ (a meno che quest’ultima transizione non scattata non sia stata disabilitata prima). *In pratica: se una transizione ha lft finito e può scattare, prima o poi deve farlo.* **Esempio classico:** $p_1 \xrightarrow{t_1 [2, \infty)} p_2 \xrightarrow{t_2 [3, enab+6)} p_3$ Gettone in p_1 al tempo 0 $\rightarrow t_1$ scatta a 4 $\rightarrow t_2$ abilitata a 4 con $lft = 10$. In STS t_2 **deve** scattare entro il tempo 10!

Teoremi fondamentali:

- Ogni sequenza ammissibile in WTS può essere riordinata in una sequenza equivalente ammissibile in MWTS $\Rightarrow$ possiamo **ignorare A2** nell’analisi (usiamo sempre la semantica debole locale).
- In virtù dell’ultimo assioma si potrebbe pensare che esista un modo per trasformare ogni sequenza di scatti MWTS in una sequenza STS, realizzando così un’equivalenza. Purtroppo, però, non è così: una sequenza STS è sempre anche MWTS, ma non è sempre vero il contrario.

Esempio pratico WTS vs STS



Timestamp iniziale: $\tau(p_1) = 0$

- t_1 abilitata alle 0 (abilitata ma potrà scattare solo a 2) $\rightarrow$ scatta (es.) alle 3 $\rightarrow$ nuovo gettone in p_2 con $\tau = 3$
- t_2 abilitata alle 3 $\rightarrow$ può scattare in $[6, 9]$
- **WTS/MWTS**: t_2 può non scattare mai (poiché non vale a5)
- **STS**: t_2 **deve** scattare entro il tempo 9

11.3 Semantica temporale mista (Mixed Time Semantics)

Quando il sistema contiene sia componenti deterministici che non-deterministici (umani che scelgono se far scattare o meno) è restrittivo usare una semantica unica per tutta la rete.

Definizione

Si assegna la semantica **a ogni singola transizione**:

- **Transizioni forti** (default): devono scattare entro il loro *lft* se restano abilitate
- **Transizioni deboli**: possono scattare in qualsiasi momento del loro intervallo (anche mai)

Convenzione grafica: transizioni deboli indicate con **W**, altrimenti forti per default.

Esempio classico

Rete:

$$p_1 \xrightarrow{t_1 [1, \infty) \text{ W}} p_2 \xrightarrow{t_2 [enab+3, 10]} p_3$$

- un gettone in p_1 creato al tempo 0
- t_1 è debole (ha la W)
- t_2 è forte (nessuna W $\rightarrow$ forte di default), $lft = 10$

fino a quando può aspettare t_1 ?

t_1 deve scattare **al più tardi al tempo 7**, perché:

- se t_1 scatta al tempo θ , allora p_2 riceve il gettone al tempo θ
- t_2 diventa abilitata al tempo θ
- t_2 è forte e deve scattare entro il tempo assoluto 10
- il suo $eft = 3$, quindi $\theta + 3$
- ma deve essere $\leq 10 \rightarrow \theta + 3 \leq 10 \rightarrow \theta \leq 7$

Se t_1 scattasse dopo il tempo 7 (es. a 8), t_2 verrebbe abilitata a 8 e il suo eft diventerebbe $8+3 = 11$, ma la transizione ha lft assoluto 10 $\rightarrow$ non potrebbe più rispettare la scadenza $\rightarrow$ violazione di A5.

12 Le reti Time Basic sono riducibili a reti ad alto livello?

è possibile tradurre una rete Time Basic in una rete ad alto livello (Colored/High-Level Petri Net) riutilizzando le tecniche di analisi già esistenti?

Tentativo di traduzione TB $\rightarrow$ reti ad alto livello

Idea:

- Ogni gettone porta una variabile **chronos** = timestamp di creazione
- I predicati controllano i timestamp e gli intervalli $[eft, lft]$
- Le azioni assegnano ai nuovi gettoni il timestamp θ di scatto

Questo funziona perfettamente per la parte sintattica.

Il problema vero: la semantica forte

Assioma	Modellabile in reti ad alto livello?	Note
A1, A3 (base)	Sì	controlli locali sui timestamp
A2 (monotonia)	Sì	posto globale “tempo corrente”
A4 e A5 (forte)	NO	richiedono conoscenza globale della marcatura

A5 è il killer: per sapere se una transizione può scattare adesso, dovrebbe sapere se **un'altra** transizione nella rete sta per perdere il suo lft . $\Rightarrow$ servirebbe che ogni predicato vedesse **tutta** la marcatura della rete $\rightarrow$ viola la località $\rightarrow$ impraticabile.

Conclusione «Le reti Time Basic **non sono riducibili** a reti ad alto livello perché la semantica forte richiede conoscenza globale dello stato, distruggendo la località tipica delle reti di Petri. Per questo sono stati sviluppati algoritmi di analisi specifici (stati simbolici + Floyd + inclusione + ecc.).»

12.1 Reti Time Petri ad Alto Livello (HLTPN)

Poiché la riduzione non è possibile, nasce il modello più potente in assoluto: **High-Level Time Petri Nets (HLTPN)** = reti ad alto livello + tempo esplicito Time Basic.

Caratteristiche principali:

- **Alto Livello:** Gli intervalli di scatto possono dipendere dal contenuto dei gettoni
- **temporizzate:** estensione che crea predicati in modo da non far scattare una transizione solo se ci sono gettoni ma anche se rispettano i vincoli.
- **reti di petri:** struttura di base su cui vengono aggiunte le estensioni

Teorema importante per l'esame La complessità dominante nell'analisi delle HLTPN proviene **sempre** dalla parte temporale. Chi sa analizzare le Time Basic sa analizzare anche le HLTPN (la parte funzionale si riduce con le tecniche classiche: unfolding, coverability graph, ecc.).

13 Analisi di raggiungibilità temporale nelle Time Basic

Nelle reti classiche il grafo di raggiungibilità è finito (o al massimo usa ω). Nelle Time Basic è **sempre infinito** per due motivi fondamentali:

1. Ogni transizione può scattare in infiniti istanti diversi $\rightarrow$ infiniti timestamp
2. Anche senza transizioni abilitate il tempo continua ad avanzare

$\Rightarrow$ serve un approccio completamente nuovo.

13.1 Stati simbolici $[\mu, C]$ – la soluzione

Invece di timestamp numerici si usano **simboli** (τ_i) + **vincoli** (C).

Uno stato simbolico è la coppia $[\mu, C]$ dove:

- $\mu(p) = \text{multiset di simboli } \tau_i \text{ in ogni posto}$
- $C = \text{insieme di disequazioni tra i } \tau_i \text{ (es. } \tau_0 < \tau_1 \leq \tau_0 + 5)$

Esempio classico

Rete con tre posti: - gettone in p_1 e p_3 creato al tempo 0 - gettone in p_2 creato al tempo 1

Stato simbolico:

$$\mu(p_1) = \tau_0, \quad \mu(p_2) = \tau_1, \quad \mu(p_3) = \tau_0$$
$$C = \{\tau_0 < \tau_1\}$$

13.2 Formula per generare il nuovo stato simbolico

Quando la transizione t scatta in uno stato $[\mu, C]$:

1. Introduci nuovo simbolo τ_{new}
2. Aggiorna la marcatura μ' (togli preset, aggiungi postset con τ_{new})
3. Calcola i nuovi vincoli C' :

Esempio pratico completo

Rete: $p_1 \xrightarrow{t[2,5]} p_2$, gettone in p_1 creato al tempo 0

Stato iniziale: $\mu_0: p_1 = \tau_0, \quad C_0: \tau_0 = 0$

Dopo lo scatto di t : $\mu_1: p_2 = \tau_1 \quad C_1: \tau_0 = 0 \wedge \tau_1 \geq \tau_0 \wedge 2 \leq \tau_1 \leq 5 \Rightarrow C_1: 2 \leq \tau_1 \leq 5$

ATTENZIONE:

ripetendo i passaggi per trovare i nuovi stati ci accorgiamo che l'algoritmo non termina mai, infatti è difficile capire se uno stato è stato esplorato o meno e non ci garantisce di esplorarli tutti, nonostante ciò questo metodo è utile per vedere come si evolve la rete e se è consistente in un certo lasso di tempo.

13.3 Dall'albero infinito al grafo finito – le 4 tecniche di contrazione

L'albero di raggiungibilità simbolico è **sempre infinito** (anche per reti banalissime). Il tempo non torna indietro $\rightarrow$ **non può esistere un grafo ciclico**, ma vogliamo almeno un **grafo aciclico finito** più compatto.

Per riuscirci dobbiamo **dimenticare la storia** di come siamo arrivati in uno stato e riconoscere che due stati simbolici diversi possono in realtà **rappresentare la stessa evoluzione futura**.

Le quattro tecniche (da sapere in quest'ordine) sono:

1. Algoritmo di Floyd – eliminare i vincoli “fantasma”

Ogni nuovo stato eredita tutti i vincoli del precedente $\rightarrow$ due stati con la stessa marcatura ma percorsi diversi hanno vincoli diversi $\rightarrow$ appaiono diversi.

Molti vincoli coinvolgono simboli τ_i **già consumati** (non più presenti). Non possiamo semplicemente cancellarli: possono esprimere vincoli **indiretti** sulle variabili ancora vive.

Esempio classico:

Vincoli: $B - A \leq 5 \quad C - B \leq 6$ dove B è morto (non più nella marcatura)

Sommando: $C - A \leq 11 \rightarrow$ vincolo tra A e C che va mantenuto!

Algoritmo di Floyd-Warshall

1. Normalizzazione dei Vincoli:

- Riscrivi tutti i vincoli temporali (derivati da durate delle transizioni o da scadenze) nella **forma canonica**:

$$T_X - T_Y \leq k$$

- Questa disuguaglianza definisce un arco diretto nel grafo dei vincoli: un arco da **Y** a **X** con peso k .
- **Esempio:** Il vincolo $T_X \geq T_Y + \tau$ (ovvero $T_X - T_Y \geq \tau$) si converte in $T_Y - T_X \leq -\tau$, creando l'arco $X \xrightarrow{-\tau} Y$.

2. Costruzione della Matrice di Adiacenza:

- Inizializza la **Matrice dei Vincoli** M , dove righe e colonne corrispondono a tutte le **variabili temporali** (istanti di scatto vivi e morti).
- Imposta $M[i, j] = k$ per ogni vincolo diretto $T_i - T_j \leq k$.
- Imposta $M[i, j] = \infty$ se non esiste un vincolo diretto tra j e i .
- Imposta $M[i, i] = 0$.

3. Esecuzione dell'Algoritmo di Floyd-Warshall:

- Applica l'equazione di Bellman, iterando su tutti i possibili nodi intermedi **k**:

$$M[i, j] \leftarrow \min(M[i, j], M[i, k] + M[k, j])$$

- **Ruolo di $M[i, j]$:** Alla fine, $M[i, j]$ conterrà il **cammino minimo** (il vincolo temporale più stringente) dal nodo **j** al nodo **i**. Questo impone la disuguaglianza $\mathbf{T_i} - \mathbf{T_j} \leq \mathbf{M[i, j]}$.
- **Verifica Cicli Negativi:** Se $M[i, i] < 0$ per qualsiasi i , il sistema di vincoli è **contraddittorio** e non ammette soluzioni ammissibili.

4. Riduzione della Matrice Finale:

- Estrai dalla matrice risultante M **soltanto le righe e le colonne** corrispondenti alle variabili temporali delle transizioni **ancora vive** e rilevanti per l'analisi futura.

Dopo Floyd due stati con stessa μ e stessi vincoli (anche se percorsi diversi) diventano **uguali**.

2. Inclusione di stati (state class inclusion)

Anche dopo Floyd possiamo avere stati identici tranne per una costante.

Esempio: rete con ciclo che aggiunge +1 ogni giro $\rightarrow$ vincoli: $\tau_1 \leq 5, \tau_1 \leq 6, \tau_1 \leq 7, \dots$

Tutti hanno la stessa μ , ma vincoli sempre più larghi.

Definizione di inclusione Uno stato simbolico $S_1 = [\mu_1, C_1]$ è **contenuto** in $S_2 = [\mu_2, C_2]$ ($S_1 \sqsubseteq S_2$) se e solo se:

- $\mu_1 = \mu_2$
- $C_1 \implies C_2$ (i vincoli di S_1 sono più restrittivi)

+ Regola del grafo: **tieni solo lo stato più generale** (quello con vincoli più larghi).

3. Tempi Relativi: Astrazione Temporale

Il principio dei Tempi Relativi (o Astrazione Differenziale) è una tecnica di riduzione dello spazio degli stati nelle Reti di Petri Temporizzate (TPN) quando l'evoluzione della rete è indipendente da un orologio globale.

Problema: Esplosione dello Stato con Tempi Assoluti

Nelle TPN, ogni stato (marcatura M) è definito da un insieme di gettoni e dai loro **timestamp assoluti** (ts). Mantenere un riferimento a un tempo assoluto t_{assoluto} causa un'esplosione dello spazio degli stati.

- Due stati che differiscono solo per un offset costante nel tempo sono considerati distinti, anche se la loro evoluzione dinamica sarà identica.
- **Esempio:** Un gettone creato al tempo $t = 0$ o al tempo $t' = 1000$ porta alla **stessa evoluzione futura** se i vincoli delle transizioni sono solo relativi.

Principio: Eliminazione dei Riferimenti Assoluti

Obiettivo: L'evoluzione futura dipende solo dalle **differenze** (o ritardi) tra i timestamp, non dal loro valore assoluto.

Tecnica di Trasformazione

Per eliminare la dipendenza dal tempo assoluto, si opera una **traslazione dello zero temporale** ad ogni scatto di transizione:

1. **Scelta del Riferimento (τ_{rif}):** Si identifica un **tempo di riferimento** τ_{rif} nella marcatura corrente, spesso il **minimo** o il **massimo** dei timestamp attuali, o il tempo del prossimo evento inevitabile.
2. **Ricalcolo dei Vincoli Relativi:** Tutti i timestamp ts_i e i vincoli temporali sono **riscritti** in relazione a τ_{rif} :

$$ts'_i = ts_i - \tau_{\text{rif}}$$

3. **Eliminazione del Riferimento a τ_{rif} :** Poiché l'evoluzione futura dipende dalle differenze, l'istante di riferimento τ_{rif} (che diventa zero nel sistema relativo) può essere ignorato.
4. **Risultato:** Stati che prima differivano solo per un offset temporale (Δt) vengono **identificati come equivalenti**. Questo riduce drasticamente il numero di stati distinti nel grafo di raggiungibilità, rendendolo analizzabile.

Esempio Pratico: La Coda Auto-Ciclica

Obiettivo: Dimostrare che la rete può evolvere all'infinito senza generare infiniti stati nel grafo di raggiungibilità, ignorando il tempo assoluto.

Rete (Implied Time Basic PN):

$$P_1 \xrightarrow{T_1 [\tau_1, \tau_2]} P_1 \quad \text{Marcatura iniziale: } M_0(P_1) = 1$$

- Il posto P_1 ha un gettone.
- La transizione T_1 consuma e produce un gettone da P_1 .
- Vincolo di T_1 : Scatta in un intervallo relativo **dopo** l'abilitazione. Ad esempio, $[1, 2]$. (Deve scattare tra 1 e 2 unità di tempo dopo che è abilitata).

Scenario 1: Analisi con Tempi Assoluti (L'approccio problematico)

1. M_0 : P_1 ha un gettone g_0 con $ts(g_0) = 0$.
2. T_1 **scatta**: T_1 è abilitata a $t = 0$. Deve scattare in $[0 + 1, 0 + 2] = [1, 2]$. Supponiamo scatti a $t_1 = 1.5$.
3. M_1 : P_1 ha un nuovo gettone g_1 con $ts(g_1) = 1.5$.
4. T_1 **scatta di nuovo**: T_1 è abilitata a $t = 1.5$. Deve scattare in $[1.5 + 1, 1.5 + 2] = [2.5, 3.5]$. Supponiamo scatti a $t_2 = 2.8$.
5. M_2 : P_1 ha un nuovo gettone g_2 con $ts(g_2) = 2.8$.

Poiché $ts(g_1) \neq ts(g_2)$, gli stati M_1 e M_2 sono considerati **distinti** nell'analisi assoluta. Qualsiasi scelta di tempo di scatto crea un nuovo stato $\rightarrow$ **Infiniti stati**.

—
Scenario 2: Analisi con Tempi Relativi (L'approccio corretto)

Si applica la tecnica **elimina i riferimenti assoluti** scegliendo il timestamp del gettone come τ_{rif} (il nuovo zero temporale).

1. M_0 : P_1 ha g_0 con $ts(g_0) = 0$. Il tempo di riferimento è $\tau_{\text{rif}} = 0$. Il gettone ha $ts' = 0$.
2. T_1 **scatta a t_1 (Relativo)**: T_1 scatta in $[1, 2]$ (ritardo Δt).
3. M_1 (**Traslazione**): Il nuovo gettone g_1 è prodotto con $ts(g_1) = t_1$. **Si trasla il tempo**: Si sceglie t_1 come nuovo τ_{rif} .

$$ts'(g_1) = ts(g_1) - t_1 = 0$$

4. **Evoluzione Futura**: Lo stato M_1 viene identificato come uno stato in cui P_1 ha un gettone con $ts' = 0$, che è **identico** allo stato iniziale M_0 .

Risultato: La rete ha solo **un unico stato nel grafo di raggiungibilità relativo** ($M_0 \equiv M_1 \equiv M_2 \dots$), dimostrando che l'evoluzione può essere infinita senza esplosione dello stato.

4. Gettoni anonimi τ_A

Se il timestamp di un gettone **non influenza più nessuna decisione futura** (non serve per calcolare nessun *eft* o *lft*), allora possiamo **cancellarlo**.

13.4 Riepilogo delle 4 tecniche (tabella da imparare)

Tecnica	Cosa elimina	Effetto sul grafo
Floyd	vincoli indiretti su simboli morti	stati con storie diverse coincidono
Inclusione	stati con vincoli più restrittivi	elimina stati “troppo stretti”
Tempi relativi	riferimenti a 0, 10, 100...	stati shiftati nel tempo coincidono
Gettoni anonimi τ_A	timestamp inutili al futuro	cicli infiniti diventano finiti